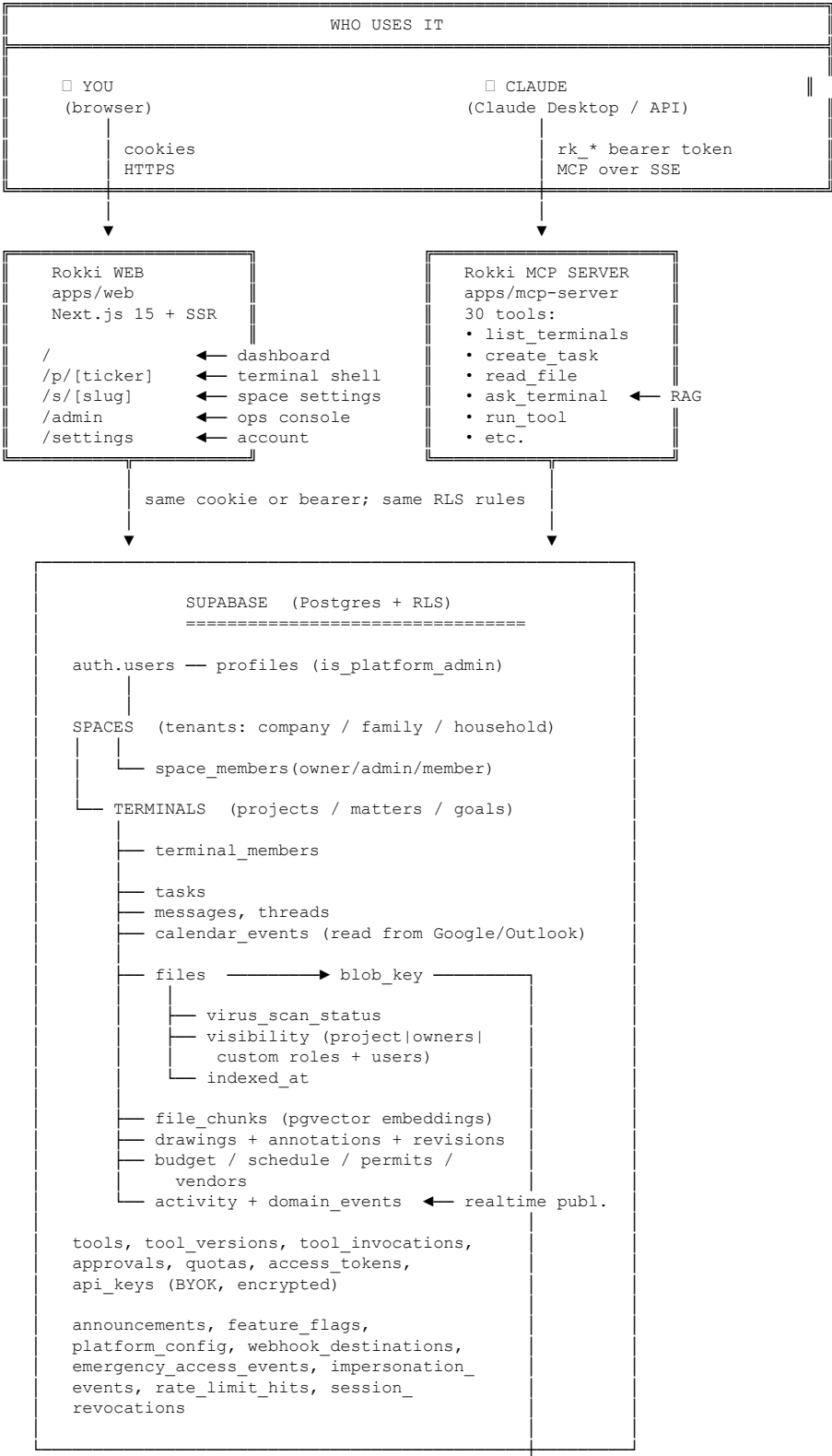


Rokki — System Architecture

A one-page map of how Claude, the web app, Postgres, blob storage, and the background workers fit together. Printable. Updated as the platform evolves.

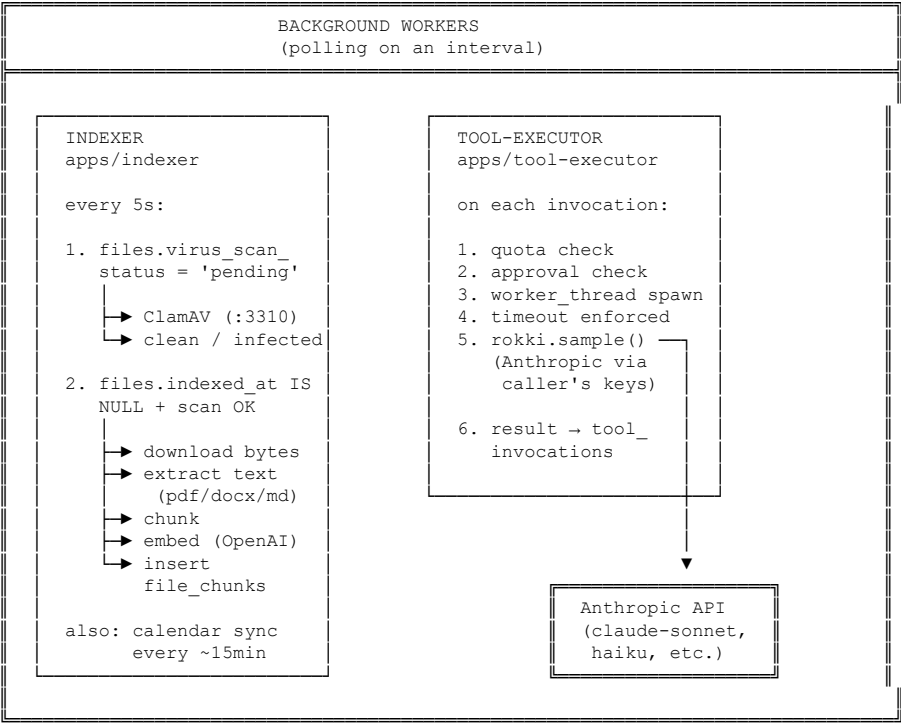
Map



| S3 API
▼

BLOB STORAGE
MinIO (local)
Azure (prod)
file bytes only

Background workers



Glossary

HOW THE PIECES NAME EACH OTHER	
SPACE	= a tenant. "Helios AI", "McKerley Family" Only platform admins create these.
TERMINAL	= one working context inside a space. A project, a matter, a client, a goal. "HLX", "BRKL", "MIA-LEGAL-2026". Space owner/admin creates these.
TASK	= a unit of work inside a terminal. Any terminal member creates these.
FILE	= a blob uploaded into a terminal, scanned, indexed for RAG.
TOOL	= a piece of code in the marketplace. Runs in the executor sandbox. Reachable from the UI (/tools), from MCP (rokki_run_tool), and from the CLI (rokki publish).
MCP	= Model Context Protocol – the open standard Claude uses to call your own data and actions. Rokki's MCP server is what makes "the terminal for your projects" work *inside Claude*.

Reading the map

1. You + Claude read the same data.

You sign in on the left, Claude connects from the right — both end up reading and writing the same Postgres, governed by the same RLS rules. Claude can't do anything in the MCP that you couldn't do in the UI, and vice versa.

2. The web app is the whole UI.

apps/web is the dashboard, terminal shell, admin console, settings. Server components pull from Postgres directly; client components call **/api/v1/*** routes.

3. The MCP server is Claude's entry point.

Same permissions model, same data. Claude Desktop points at the SSE URL, uses your **rk_*** token, and calls **rokki_*** tools.

4. Postgres is the hub.

Spaces contain terminals contain files/tasks/etc. Files live as *metadata* in Postgres (with **blob_key** pointing to S3) and as *bytes* in MinIO/Azure.

5. Two background workers chew on the queue.

The **indexer** scans for viruses then embeds text for RAG. The **tool-executor** runs user-published tools in sandboxed Node workers.

6. How Claude answers about a terminal.

When Claude calls **ask_terminal**, the MCP server runs hybrid RRF search over **file_chunks**, returns the best passages as context, and Claude answers using them.